Vaibhav Kumar · Design & Frontend.

ARRIVO · INNOTECH'26 · CATEGORY 6 · YOUR STUDY GUIDE

Your part: everything a person sees and touches on the Arrivo site:

- the look (colours, fonts, dark mode)
- the page itself
- the booking panel and the bay pass
- the list of all 96 bays
- how the screen stays live while other people book

The rest of the team covers the other parts:

- **Kritika:** the lot plan, the lens and the 3D view
- **Prisha:** the Python server and the database
- **Krish:** the problem and the impact

You only need to know their parts well enough to hand over to them.

Your slides:

- **6:** How a booking works
- **8:** only the dark-mode card
- **9:** Booking → pass
- **10:** the live demo (you drive the mouse)
- **11:** Tech stack: the Languages, Frontend, Design and Tools rows. Prisha takes the Backend row.

How to use this guide. Read it twice. Then open every file it names in a code editor (VS Code is fine) and find each snippet yourself. If you can point at the code, you can explain it.

1. Say this

These are short scripts you can learn almost word for word. Say them in your own voice.

> **Slide 6 · How a booking works.** "Booking with Arrivo takes five steps. The lot is drawn once from its satellite photo. Every bay gets a name, a size and a type. You pick when you arrive and how long you stay. Hours that are already taken show grey, like seats someone else bought. You confirm, and the bay is held in your name with a pass and a code. Then you drive straight to it, and it is free, because nobody else could book it. Step four is what makes us different: we don't just *show* free space, we *hold* it for you."

Slide 8 · dark mode. "The whole site has a dark theme, one switch in the top bar. It remembers your choice, and on your first visit it follows your phone's own setting."

Slide 9 · Booking. "This is the booking panel. Pick an arrival time and how long you stay, then type your number plate. The total shows before you confirm. Times that clash with someone else's booking are crossed out, so you can't pick them. You get a yellow bay pass with a code to show at the gate. If you're late, the bay waits 15 minutes for you, then it opens up for someone else."

Slide 11 · Tech stack. "The page is plain HTML, CSS and JavaScript. Only the parts that change live are written in React: the plan, the booking panel and the list. Vite builds it, Motion animates it and Lenis makes the scrolling smooth. That's about 7,400 lines of frontend code. Prisha will tell you about the backend."

2. The site in one picture

```
THE BROWSER
 index.html ............ the page: top bar, menu, hero text, sections, footer   (plain HTML)
   page.css ............ colours, fonts, light/dark theme, layout                 (plain CSS)
   page.js ............. smooth scroll, theme switch, menu, scroll effects        (plain JS)
   three empty boxes that React fills in:
     #seatmap-root ..... the seat map at the top            -> HeroSeats   (LiveLot.jsx)
     #how-root ......... the "How a booking works" drawing  -> HowItWorks  (HowItWorks.jsx)
     #live-root ........ live plan + bay panel + bay list   -> LiveLot     (LiveLot.jsx)

        |  asks for data (fetch)          ^  live changes pushed down (Server-Sent Events)
        v                                 |

THE SERVER  (Prisha's part: backend/server.py, Python, port 8787)
   GET  /api/lots/kiet-main/snapshot   -> every booking for today
   POST /api/bookings                  -> make a booking (refused if it clashes)
   GET  /api/lots/kiet-main/stream     -> stays open; sends each change as it happens
```

Why the page is half plain HTML and half React

The text, headings and sections never change, so they are plain HTML. Plain HTML has three advantages:

- it shows up as soon as the page loads
- it needs no library
- anyone on the team can edit it

The three live parts are different. They change every few seconds when someone books, arrives or leaves. At those moments, all of these must agree with each other:

- the 96 bays on the plan
- the open panel
- the list of every bay
- the counters in the top bar

Keeping all of that in sync by hand is where bugs come from. React redraws every one of them from **one shared state**. This pattern is called **islands**: a static page with small interactive "islands" inside it.

How the two halves talk

React tells the plain page the live numbers by firing a browser event called `lot:update`. `page.js` listens for that event and updates the counts in the top bar, the menu and the footer. See `main.jsx` lines 18–36 and `page.js` lines 183–210.

3. The files

FILE	LINES	WHAT'S INSIDE
`frontend/index.html`	507	The whole page. Three empty `<div>`s (lines 139, 179, 216) are where React draws.
`frontend/src/styles/page.css`	1,682	The colours and fonts, light and dark themes, layout, buttons, top bar, menu, footer, and the phone rules.
`frontend/src/page.js`	210	Plain JavaScript: smooth scroll, theme switch, menu, word-by-word headings, scroll reveals, the dot on the right edge, live counters.
`frontend/src/main.jsx`	54	Where React starts. It puts each island into its box and runs the bridge that sends the numbers to `page.js`.
`frontend/src/LiveLot.jsx`	892	Hero seat map, bay panel, booking form, bay pass, live feed, list of all bays. **Your biggest file.**
`frontend/src/HowItWorks.jsx`	307	The five-step drawing that plays as you scroll.
`frontend/src/ui.jsx`	189	Small shared pieces: the pill button, handwritten notes, rolling numbers, the day strip, status dots, bay-type icons.
`frontend/src/styles/islands.css`	2,031	Styles for the React parts. The map styles in it are Kritika's.
`frontend/src/lot.jsx`	595	The data layer. The top half (bay geometry) is Kritika's. You need the bottom half: the status rules, the server calls and the live state.
`frontend/src/LotMap.jsx`	933	**Kritika's.** The plan, the 3D view and the lens.
`frontend/package.json`	—	The list of libraries and the commands (`npm run dev`, `npm run build`).

FILE	LINES	WHAT'S INSIDE
`frontend/vite.config.js`	—	Vite settings: forwards `/api` to the Python server while developing.

Frontend total: 507 + 210 + 1,682 + 2,970 (React files) + 2,031 = **7,400 lines**, which is the number on slide 11.

4. Everything we used: what it is and why

Languages

LANGUAGE	WHAT IT IS	WHERE WE USE IT
HTML	The structure of a web page: headings, buttons, sections.	`index.html`. Everything that doesn't change.
CSS	How the page looks: colours, sizes, layout, animation.	`page.css`, `islands.css`.
JavaScript	The language browsers run. Makes the page react to clicks and scrolling.	`page.js`, and everything React does.
JSX	JavaScript with HTML-like tags inside it. It is React's way of writing UI. Files end in `.jsx`. The browser can't read it directly; Vite turns it into plain JavaScript.	`main.jsx`, `LiveLot.jsx`, `HowItWorks.jsx`, `ui.jsx`, `lot.jsx`, `LotMap.jsx`.
SVG	Pictures described as shapes in code: lines, rectangles, paths. They stay sharp at any size, and CSS can colour them, so they follow the dark theme automatically.	The lot plan, the How-it-works drawing, every icon, the barcode on the pass.

Libraries (from `package.json`)

LIBRARY	VERSION	WHAT IT IS	WHY WE CHOSE IT
React + react-dom	19.2	A library for building UI from *components*: functions that return what the screen should look like for the current data. When the data changes, React updates only what changed.	The live parts have one source of truth: today's bookings. React redraws the plan, panel, list and counters from it, so they can never disagree.
Vite	8.3	A *dev server* and a *bundler*. While coding, `npm run dev` serves the site and updates the browser the moment you save a file (*hot reload*). `npm run build` packs everything into a few small files in `dist/`.	It's fast, and it's the standard way to start a React project now. Create React App, the old way, was officially deprecated in 2025. It also understands JSX and npm imports without extra setup.
@vitejs/plugin-react	6.1	Vite's React add-on: turns JSX into JavaScript and keeps component state while hot reloading.	Needed for React in Vite.

LIBRARY	VERSION	WHAT IT IS	WHY WE CHOSE IT
Motion (`motion/react`)	14.3	An animation library for React, formerly called *Framer Motion*.	CSS alone can't animate something *leaving* the screen, can't move a highlight from one button to another, and can't do drag. Motion does all three: `AnimatePresence` for exits, `layoutId` for the sliding switch, `drag` for the phone bottom sheet.
Lenis	1.3	A small smooth-scrolling library. Instead of jumping in steps, the page glides.	It gives the calm, premium feel of good portfolio sites. All in-page links glide to their section with it too. We switch it off for people whose system asks for reduced motion.
Fontsource (`@fontsource/...`)	5.3	Google Fonts packaged as npm installs. The font files ship *inside* our site.	No call to Google's servers, so it works on bad venue Wi-Fi. No third party sees our visitors. The fonts load from our own server.
oxlint	1.81	A *linter*: a tool that reads code and warns about mistakes, such as unused variables or a typo in a name. Very fast (written in Rust).	Catches bugs before they reach the browser. Run it with `npm run lint`.

Things built into the browser (no library needed)

FEATURE	WHAT IT DOES FOR US	WHERE
CSS custom properties (variables like `--ink`)	Every colour is a variable. Dark mode just swaps the variables.	`page.css` lines 11–87
localStorage	Saves your theme choice in the browser.	`index.html` 14–25, `page.js` 51–53
matchMedia	Reads "does this system prefer dark?" and "does this person want reduced motion?", and checks screen width for phone layouts.	`index.html` 19, `page.js` 7, `ui.jsx` 179
IntersectionObserver	Tells us when an element scrolls into view, without checking on every scroll. That's how things fade in once.	`page.js` 118–130
requestAnimationFrame	Runs code right before each screen refresh (about 60 times a second), for smooth number counting and the scroll dot.	`page.js` 149–197
fetch	Sends requests to the server (load the day, make a booking).	`lot.jsx` 314–338
EventSource (*Server-Sent Events*)	Keeps one connection open so the server can push each change the moment it happens.	`lot.jsx` 343–384
CustomEvent	Our own browser event, `lot:update`, to pass numbers from React to `page.js`.	`main.jsx` 33
position: sticky	Keeps the How-it-works drawing pinned while you scroll through its five steps.	`islands.css` 11–20

FEATURE	WHAT IT DOES FOR US	WHERE
backdrop-filter	The frosted-glass top bar after you scroll.	`page.css` 612
clamp()	Font sizes that grow with the screen but never go below a minimum or above a maximum.	`page.css` 176, 184 and more

Outside tools

TOOL	WHAT WE USED IT FOR
Node.js + npm	Node runs JavaScript outside the browser. npm installs the libraries above and runs our commands. Only needed while building; visitors never need it.
Google Earth	The satellite capture of the KIET lot that the plan is traced from (Kritika's part).
Cloudflare Tunnel	Gives a temporary public web address for the site running on our laptop, so friends could open it on their phones.
metic.studio	The design studio website we used as our style reference: type scale, grid lines, calm motion. We studied the feel; we didn't copy their code.

5. The design

Colours

NAME	LIGHT	DARK	USED FOR
Paper	`#f4f4f1` warm off-white	`#060608` near-black	Background
Ink	`#121214` soft black	`#f0f0f3` soft white	Text, booked bays, dark buttons
Sun	`#ffd75a` soft yellow	`#ffd75a` (same)	The one accent: your bay, the main button's icon, the live dot, the pass
Sun ink	`#8a6400` dark gold	`#f3cf6b`	Yellow-family text that stays readable (handwritten notes, labels)
Greys	`--ink-2` … `--ink-5`, `--paper-2/3`	swapped	Secondary text, lines, empty bays

Why these colours:

- **One accent only.** Yellow is the colour of road and parking markings. Using just one accent means the eye goes to what matters: the bay that is free, or yours.
- **No pure black or pure white.** Text is never `#000` on `#fff`. Soft black on off-white is easier on the eyes and looks printed.
- **Darker dark mode.** The dark background was made extra dark on purpose. It looks deliberate and saves battery on OLED phones.

Fonts

FONT	USED FOR	WHY
Bricolage Grotesque (variable)	Big headings, bay names like `A-14`	Has character but stays clean. *Variable* means one font file holds every weight.
Space Grotesk	Body text	Very readable, with a slightly technical feel that fits "measured, drawn lot".
IBM Plex Mono	Labels, times, counters	*Monospaced*: every digit is the same width, so `012 → 013` doesn't make the text jiggle.
Caveat	Handwritten notes ("pale ones are free. tap one.")	Feels like notes scribbled on a plan, and makes the site feel human.

The visual ideas we repeat

- **A cinema seat map.** The very first thing you see is rows A and C drawn like seats. It *is* the pitch: "book a bay like a seat".
- **A design-tool look.** These say "this lot was measured and drawn", and it really was, from the satellite photo:
  - a pixel ruler on the left edge
  - white selection handles on the corners of boxes
  - size labels like `Button · 196 × 58`
- **Handwritten notes with arrows that draw themselves.** They point at what to do next.
- **Numbers padded to three digits** (`012 free`) in the mono font, like a scoreboard.
- **One easing curve everywhere:** `cubic-bezier(0.22, 1, 0.36, 1)`, a fast start and a very soft stop. Using one curve is why every motion on the site feels like it belongs together.

How dark mode works (be able to explain this exactly)

1. **Before the page shows.** A tiny script at the top of `index.html` (lines 14–25) picks the theme. It uses your saved choice from `localStorage` if there is one, otherwise your system setting (`prefers-color-scheme`). It writes the result as `data-theme="dark"` or `"light"` on the `<html>` tag. Because this runs *before* the page is drawn, there is no white flash.
2. **The CSS swaps the colours.** `page.css` has the light colours under `:root` (line 11) and the dark ones under `:root[data-theme='dark']` (line 57). Every colour in the site is written as a variable, like `color: var(--ink)`. So changing the variables recolours *everything* at once: text, bays, SVG drawings, the plan.
3. **The switch saves your choice.** The switch in the top bar (`page.js` lines 38–56) flips `data-theme` and saves your choice in `localStorage` as `arrivo.theme`.

```
// page.js (simplified)
themeBtn.addEventListener('click', () => {
  const next = root.dataset.theme === 'dark' ? 'light' : 'dark'
  root.dataset.theme = next
  localStorage.setItem('arrivo.theme', next)
})
```

One exception: the bay pass stays yellow with dark text in both themes, like a real printed ticket.

6. The page, top to bottom

SECTION	WHAT IT DOES	HOW IT'S BUILT
Top bar	Logo, five little bars, Look closer (jumps to the plan), "N free now", Book a bay, theme switch, menu.	`index.html` 29–74. The five bars next to the logo are the five rows; each bar's height is the share of that row that's free right now (`page.js` 204–206). After you scroll 24 px it turns frosted glass.
Menu	Full-screen menu with five chapters and the live free count.	`page.js` 58–84. Escape closes it. Smooth scroll pauses while it's open. Items slide in one after another.
Right-edge dot	A dot that travels down the right edge as you scroll and shows the chapter name.	`page.js` 140–181. The dot eases 14% of the way to its target every frame, which makes it glide.
Hero	The seat map of rows A and C (live), the line "Park like you booked a seat.", two buttons, and the lot facts.	The seat map is `HeroSeats` in `LiveLot.jsx` 32–82. Tapping a seat selects that bay and scrolls you to the plan.
How a booking works	A drawing that builds itself as you scroll: lot, bay with its size, other cars and a time window, the hold, then your car drives in.	`HowItWorks.jsx`. The section is 520% of the screen tall. Inside it, a sticky box stays put. Your scroll position becomes a number from 0 to 1, and that number drives the drawing.
Live lot	Big Free / Booked / Reserved counters, the plan (Kritika's), the bay panel, the legend.	`LiveLot` in `LiveLot.jsx` 826–892. The Plan/3D switch and row zoom A–E are here; the map itself is `LotMap.jsx`.
Bay panel	Empty state: how to read colours, 6 free bays closest to the campus road, and a live feed of the last 6 events. With a bay picked: details and the booking form.	`SlotPanel` 507–576, `PanelIdle` 475–505, `SlotDetail` 227–441.
All bays	The list of all 96 bays. Filter by status or row, sort, search. Each row shows type, status, booking, a mini day strip, rate and a Book button.	`SlotList` 671–809. A row flashes yellow for 1.4 s when its status changes. Search ignores the dash, so `c12` finds `C-12`.
Principles	Four promises: no circling, one bay one booking, live not guessed, a grace window.	Plain HTML, 221–264.

SECTION	WHAT IT DOES	HOW IT'S BUILT
Who it is for	Three cards (Drivers, Campuses, Forecasts). The one you hover or tap opens.	Plain HTML + `page.js` 132–138.
Your turn	One big "Book a bay" button drawn like a selected design element.	Plain HTML, 418–459.
Footer	Logo, chapters, contact, live free count, Google Earth credit.	Plain HTML, 463–499.

7. The booking flow, step by step

This is the part judges care about most. Know every step.

1. **Pick a bay.** You can pick from five places:
   - the hero seat map
   - the plan
   - the list
   - the quick chips
   - the live feed

   All of them call the same function, `select(id)`, which stores the chosen bay in the shared state. On a laptop the panel on the right shows it. On a phone a bottom sheet slides up (screens 980 px wide or less). On the phone you can drag it down or tap outside to close it.
2. **The panel shows the facts:**
   - row and name, e.g. "Row A · Ground Edge"
   - the bay ID, e.g. A-14
   - a status pill
   - size in metres, measured from the satellite photo
   - type
   - hourly rate
3. **The status block** says what's happening, in words:
   - "Free now. Stays free till 16:30, that is 2 h 10 m from now."
   - "Booked till 15:00", with a countdown and a progress bar
   - "Reserved for 14:30", which says the bay is held until 14:45
4. **Today strip** (06:00 to 22:00):
   - grey blocks are other people's bookings
   - a line marks *now*
   - your chosen window glides along in yellow
   - if your window would clash, it turns striped

5. **01 Arrive.** Four choices:
   - **Now** (rounded up to the next 5 minutes)
   - +30 min
   - +1 h
   - +2 h

   If the bay is occupied right now, the choices start when the current booking ends. Times that would clash are **crossed out** and can't be tapped.
6. **02 Stay.** 1, 2, 3, 4, 6 or 8 hours. Lengths that would run into the next booking are crossed out. If you change the arrival time and your length no longer fits, it switches to the longest one that does.
7. **03 Vehicle.** Your number plate. It's turned into capitals as you type (14 characters at most) and needs at least 6 letters or digits.
8. **Summary.** For example: `14:00 → 16:00 · 2 h × ₹20 = ₹40`, booking fee `₹5`, **total ₹45**.
9. **Press "Hold A-14 · ₹45".** The button says "Holding A-14…" with a progress bar for 1.1 seconds. That pause is deliberate, so the person sees something is happening. Then the browser sends the booking to the server (`POST /api/bookings`).
10. **The server has the last word.** If the window is still free, it saves the booking and replies with it. If someone else got there first, it replies **409 Conflict**. The panel then shows: *"A-14 was just booked 14:00–16:00. Pick another window."* Nothing is booked and nothing is charged. (How the server guarantees this is Prisha's part.)
11. **The pass appears.** "Held for you. show this at the gate." See section 8.
12. **Everyone else's screen updates within about a second**, through the live stream. The bay turns yellow for you and grey or hatched for others, the counters change, and a line appears in the live feed.

The form's submit code (`LiveLot.jsx` 254–271), in short:

```
const submit = (e) => {
  e.preventDefault()
  const p = plate.trim().toUpperCase()
  if (p.replace(/\s/g, '').length < 6) return setErr('Add your vehicle number, e.g. UP14 AB 1234.')
  if (clash) return setErr(`That window clashes with a booking at ${fmtTime(clash.start)}.`)
  setPhase('holding')
  setTimeout(async () => {
    const r = await book(id, start, end, p)   // goes to the server
    if (r.error) { setErr(r.error); setPhase('form') }
    else { setMade(r.booking); setPhase('done') }   // shows the pass
  }, 1100)
}
```

Prices

TYPE	RATE	WHICH BAYS
Standard	₹20 / hour	most bays
Compact	₹15 / hour	all of row B (under the trees)
EV charging	₹35 / hour	C-01 to C-04
Step-free	₹20 / hour	D-01, D-02, A-33
Booking fee	₹5 per booking	added to every booking

The prices live in `lot.jsx` lines 77–82 (`TYPES`) and `LiveLot.jsx` line 86 (`FEE`).

8. The bay pass

ON THE PASS	WHERE IT COMES FROM
"Arrivo · bay pass · KIET main lot"	fixed text
The bay ID, large (`A-14`)	the bay
Arrive / Leave	the window you picked
Vehicle	your plate (shown in full only to you)
Row	e.g. `A · Ground Edge`
Date, Paid	the day, and hours × rate + ₹5
Barcode + code like `PS-7KQ2M`	the code is made by the server

- **The code.** It's five characters chosen from a set without `0/O` and `1/I`, so it's easy to read out at the gate. "PS" is left over from the project's first name, ParkSeat.
- **The barcode.** It is drawn from the code with a small hash function (`LiveLot.jsx` 102–117). The same code always gives the same bars. **Be honest: it's for looks and isn't scannable yet.** The guard checks the code. QR check-in is on the roadmap.
- **The animation.** The pass flips down like a ticket coming out of a machine: it rotates from −70° to 0° around its top edge, with 3D perspective (`Ticket`, 119–172).
- **It survives a refresh.** The server remembers which browser made the booking, using an anonymous cookie (no login). So if you reopen that bay, even in another tab, you see your pass again.

9. Bay colours and the exact rules

STATUS	LOOKS LIKE	RULE
Free	pale, empty	Nobody is parked now and nobody is due in the next 30 minutes.

STATUS	LOOKS LIKE	RULE
Booked	grey, with a car	Someone's booking is running now.
Ending soon	grey with a **yellow ring**	The current booking ends within **20 minutes**.
Reserved	**hatched** (diagonal stripes), dashed edge	Someone booked it and is due within **30 minutes**. It waits **15 minutes** past their arrival time, then opens up if nobody shows.
Yours	**yellow**, with your car	Booked from this browser.

The rules are one small function, `slotState` in `lot.jsx` line 240. The server uses the same rules, so both sides always agree.

```
if (current) status = current.mine ? 'mine' : current.end - now <= ENDING_SOON ? 'ending' : 'taken'
else if (next && next.start - now <= HOLD_AHEAD) status = next.mine ? 'mine' : 'held'
else status = 'free'
```

Colour is never the only signal. Booked bays have a car, ending ones a ring, reserved ones stripes. Someone who is colour-blind can still read the plan.

10. How the screen stays live

All of this lives in `LotProvider` (`lot.jsx` 451–592). React's *context* shares it with all three islands, so there is **one** connection and **one** copy of the data for the whole page.

1. **Open the stream, then load the day.** On page load we open the live stream (`EventSource`), then fetch today's *snapshot*: every booking from midnight to midnight.
2. **Changes that arrive during the load wait their turn.** They go into a queue and are replayed on top of the snapshot, so no change is lost.
3. **One bay at a time.** Each change from the server is one booking as it now stands. We update just that bay's list. Cancelled bookings drop out.
4. **Two clocks:**
   - `now` ticks every **1 second**, for countdowns and the clock.
   - `tick` ticks every **5 seconds**, for bay statuses.

   Why two? Re-working out 96 bays every second is wasted work, and a status never needs to be more precise than 5 seconds.
5. **If the connection drops:**
   - We reconnect after 1 s, then 2, 4, 8, up to 15 s between tries.
   - The server sends a `ping` every 15 seconds. If we hear nothing for **40 seconds**, we assume the connection is dead and reconnect.
   - Every reconnect reloads the snapshot, so nothing is missed.

6. **Bridge to the plain page.** `Bridge` in `main.jsx` fires `lot:update` with the free, booked and reserved counts and each row's free share. `page.js` counts the numbers up smoothly in the top bar, menu, footer and Live-lot counters.

Why Server-Sent Events, and not polling or WebSockets?

- **Polling** means asking the server "anything new?" every few seconds. It's wasteful, and the screen lags.
- **WebSockets** are two-way and need more code on both sides. We only need the server to talk *to* the browser.
- **Server-Sent Events** are one-way (server → browser), built into every browser, reconnect by themselves and run over normal HTTP. That's exactly what a live map needs.

A real bug we fixed: while developing, the stream could silently die behind Vite's proxy, and the page never noticed. We added the 15-second ping, the 40-second watchdog, and a fix in `vite.config.js` that closes the stream properly when the server restarts.

11. Every animation, and how it's made

WHAT YOU SEE	HOW IT'S MADE
Headings slide up word by word	`page.js` 95–116 wraps each word in a hidden "mask". CSS moves each word up from below with a small delay per word. It starts when the heading scrolls into view.
Blocks fade up out of a blur	The `.reveal` class (opacity 0, moved down, blurred). When on screen, `.seen` is added and CSS transitions it to normal (`page.css` 278).
Smooth scrolling	Lenis (`page.js` 5–14).
Top bar drops in, then turns frosted	A CSS keyframe animation, then `backdrop-filter: blur(14px)` after 24 px of scroll.
Button arrow swap on hover	Two arrows: one flies out to the top-right while the other comes in from the bottom-left (`page.css` 296–351).
Pulsing live dot	A ring grows and fades every 1.8 s (CSS `@keyframes live-ping`).
Numbers counting up	`page.js` `countTo` : 0.9 s, slowing at the end. The React parts use `Ticker` in `ui.jsx` (Motion's `animate`).
Seats pop in one by one	Motion, with a 0.022 s delay per seat.
How-it-works drawing	Scroll position → a number 0–1 → lines "draw" using `strokeDashoffset` , and the car drives in during step 5.
Plan / 3D switch pill slides across	Motion `layoutId` : the same pill moves to the new button.
Panel content cascades in	Motion `staggerChildren: 0.06` s.

WHAT YOU SEE	HOW IT'S MADE
Switching bays in the panel	`AnimatePresence mode="wait"`: the old content fades out before the new comes in.
Phone bottom sheet	Motion `drag="y"`. It closes if dragged down more than 120 px.
Yellow window gliding on the day strip	Motion animates its `left` and `width`.
Bay pass flips down	`rotateX` from −70° with perspective.
List row flashes when a bay changes	The `is-flash` class for 1.4 s.
Live feed lines drop in	`AnimatePresence` + `layout`.
Hand-drawn arrows draw themselves	Motion `pathLength` 0 → 1 (`Hand` in `ui.jsx`).

Reduced motion: if someone's phone or computer is set to "reduce motion", Lenis is turned off and every CSS animation and transition is cut to almost zero (`page.css` 1674). Some people get dizzy from motion; this respects their setting.

Why the animations stay smooth: we mostly animate `transform` and `opacity`. The graphics chip can move those without redrawing the page. The 96 list rows are wrapped in `memo`, so only the rows whose bay changed are redrawn.

12. Phones and accessibility

- **Fluid sizes.** Big headings use `clamp()`, so they scale smoothly from phone to big screen.
- **Breakpoints.** The layout changes at 1180, 1100, 980, 860, 760, 700, 600 and 520 px wide.
- **Phone panel.** At 980 px and below, the bay panel becomes a **bottom sheet** with a dark backdrop. Tap outside or drag it down to close.
- **Inner scrolling.** The panel and the list scroll on their own (`data-lenis-prevent`), so the page doesn't scroll away under your finger.
- **Real buttons.** Every clickable thing is a real `<button>`, so it works with a keyboard.
- **Screen-reader labels.** The theme switch has `role="switch"` and `aria-checked`. The menu button has `aria-expanded`. Every seat has a label like "A-14, Free". Decorative drawings are hidden from screen readers (`aria-hidden`).
- **Keyboard.** Escape closes the menu and the bay panel. List rows can be reached with Tab and opened with Enter or Space.
- **Headings.** Split headings keep the full sentence as an `aria-label`, so screen readers don't read them word by word.

13. Run it, and the live demo

Before you present (the night before and again in the morning):

```
cd frontend
npm install
npm run build
```

```
cd backend
python server.py
```

Then open `http://localhost:8787`. The Python server also serves the built site, so that one address is the whole thing. No internet is needed: the fonts are bundled and everything runs on the laptop.

While coding you would run `npm run dev` in `frontend` instead. It serves on `http://localhost:5173`, and Vite forwards `/api` to the Python server.

Demo script for slide 10:

1. Press **L** on slide 10. The real site opens inside the deck.
2. Point at the **live count** in the top bar: "this number is live."
3. Tap a **pale (free) seat** in the hero seat map. The page glides to the plan and the panel opens.
4. Choose **Now**, **2 h**, type `UP14 AB 1234`, and press **Hold**. Show the **pass**.
5. **Hand over to Kritika.** She picks up the **lens** and drags it over the plan, then switches to **3D**.
6. Flip **dark mode** with the switch in the top bar.
7. **Double-booking test (hand over to Prisha):**
   - Open a **private window** (Ctrl+Shift+N). It counts as a different person.
   - Put the two windows side by side with the same bay and the same time picked.
   - Book it in one window.
   - Within a second, the other window's time strip turns striped and its Hold button switches off.
   - Prisha explains how the server refuses the second booking even if both press at the same instant.
8. Press **Esc** to come back to the slides.

Checklist:

- [ ] Browser zoom at 100%.
- [ ] Theme set to light before you start.
- [ ] Other tabs closed.
- [ ] Laptop charging.
- [ ] Practise the demo three times.

14. Honest limits

Know these before a judge finds them. Saying them first earns trust.

- **No real payment yet.** "Hold" reserves the bay; UPI payment is next on the roadmap.
- **The barcode isn't scannable yet.** The code is what the gate checks. QR check-in is planned.
- **The 15-minute grace is a rule we show, not yet enforced.** Without check-in, the server can't tell whether a driver arrived, so a no-show bay stays booked until its end time.
- **No login.** Your browser gets an anonymous ID in a cookie. If you clear cookies or switch browsers, the site no longer knows that booking is yours.
- **No cancel button on screen yet.** The cancel function exists in `lot.jsx` and the server supports it; it just isn't wired to a button.
- **Most cars you see are demo traffic.** The server makes them up so the lot looks alive. Bookings made on the site are real and saved in the database. `python server.py --no-demo` turns demo traffic off.
- **One lot so far** (KIET main lot).
- **No automated tests yet.** We tested by hand on laptop and phone.

15. Questions to prepare

Practise saying these out loud. Short answers first; add detail only if they ask.

Basics

Q1. What did you build the frontend with? HTML, CSS and JavaScript for the page, and React for the three live parts. Vite builds it, Motion does the animations, Lenis does smooth scrolling.

Q2. Why not React for the whole page? Most of the page never changes, so it doesn't need React. Plain HTML shows instantly and anyone can edit it. React is only where the data changes every few seconds.

Q3. Then why not plain JavaScript for everything? When one booking happens, about five places on screen must change together: the plan, the panel, the list, the counters and the feed. Doing that by hand is where bugs come from. React redraws all of them from one shared state, so they never disagree.

Q4. What is Vite? A tool that serves the site while we code, updating the browser instantly when we save. It also packs everything into a few small files for the real site with `npm run build`.

Q5. Why Vite and not Create React App or Next.js? Create React App was officially deprecated in 2025, and Vite is the standard now. Next.js is built for sites with many pages rendered on a server. We have one page, and our server is Python, so Next.js would add weight we don't need.

Q6. What's the difference between `.js` and `.jsx`? `.jsx` lets you write HTML-like tags inside JavaScript. That's how React describes what the screen should show. Vite converts it to normal JavaScript.

Q7. What is a component? What is state? A component is a function that returns a piece of UI, like `Ticket` or `SlotPanel`. State is data that can change, like which bay is selected or the plate you typed. When state changes, React re-runs the component and updates only what changed on screen.

Design

Q8. Why yellow? It's the colour of road and parking markings. We use only one accent so your eye goes to what matters: free bays, your bay and the main button.

Q9. Why these fonts?

- Bricolage Grotesque for headings, because it has character.
- Space Grotesk for reading.
- IBM Plex Mono for numbers, so digits don't jump when they change.
- Caveat for the handwritten notes.

All four are bundled with the site, so there's no dependence on the internet.

Q10. How does dark mode work? Every colour is a CSS variable. Dark mode swaps the variables. A tiny script picks the theme before the page appears, so it doesn't flash. Your choice is saved in the browser, and the first visit follows your phone's setting.

Q11. Where did the design come from? We studied design studio sites, especially metic.studio, for the type, the grid lines and the calm motion. The seat-map idea is ours: it's literally our pitch, "book a bay like a seat".

Q12. Is it responsive? Yes. Sizes scale with the screen. Below 980 px the bay panel becomes a bottom sheet you can drag closed. Show it on your phone if they ask.

Live updates

Q13. How does the map update without refreshing? The page keeps one connection open to the server, called Server-Sent Events. When anything happens, the server pushes that one change and we update just that bay. Everyone sees it within about a second.

Q14. Why not refresh every few seconds? That's called polling. It's wasteful and always a few seconds late. With Server-Sent Events the server tells us the moment something changes.

Q15. What if the internet drops? The page reconnects by itself, waiting 1, 2, 4… up to 15 seconds between tries. After reconnecting it reloads the whole day, so nothing is missed. If the server is silent for 40 seconds, we treat the connection as dead and reconnect.

Q16. What stops two people booking the same bay? On screen, clashing times are crossed out. But the real guarantee is on the server: it checks again and refuses the second booking with a "409 Conflict". Then our panel says "A-14 was just booked, pick another window". Prisha can explain the database rule behind it.

Booking and the pass

Q17. Walk me through a booking. Tap a bay, pick when you arrive, pick how long, type your plate, see the total, then press Hold. The server confirms and you get a pass with a code. Everyone else's screen updates.

Q18. Is the payment real? Not yet. Hold reserves the bay. UPI payment is the next step on our roadmap.

Q19. Does the barcode work? It's generated from the booking code, so every pass looks unique. It isn't scannable yet; the gate checks the code. QR check-in is planned.

Q20. How does the site know which booking is mine without a login? The server gives each browser an anonymous ID in a cookie. Bookings made from that browser come back marked as yours. Other people's number plates are shown masked, like `UP14 •• 1234`.

Q21. What if I'm late? The bay waits 15 minutes past your arrival time; after that it opens up for someone else. In the prototype that's the rule we show. Freeing a no-show automatically needs check-in at the gate, which is next on the roadmap.

Quality

Q22. How big is the site? Is it fast?

- The built JavaScript is about 144 KB compressed, and the CSS about 13 KB.
- Animations mostly move `transform` and `opacity`, which the graphics chip handles cheaply.
- The list rows only redraw when their own bay changes.

Q23. Is it accessible?

- Real buttons, so it works with a keyboard.
- Labels for screen readers.
- Escape closes panels.
- Colour is never the only signal: there are cars, rings and stripes too.
- We turn animations off for people whose system asks for reduced motion.

Q24. What was hard? Keeping the live connection honest. While developing, the stream could silently die behind the dev proxy and the page wouldn't notice. We added a ping every 15 seconds and a 40-second watchdog that reconnects.

Q25. How many lines of code? About 7,400 lines of frontend, plus 1,121 lines of Python on the backend.

Q26. What would you add next?

- UPI payment
- QR check-in at the gate
- a cancel button
- login, so bookings follow you across devices
- making it installable on phones (a PWA)

Process and honesty

Q27. Are those real cars on the map? The bays are real, traced from the satellite photo. The cars moving around right now are demo traffic made by our server, so the lot looks alive. Any booking you make is real and saved.

Q28. Did you use AI? Answer truthfully. If you did, say so plainly and show you understand the result. For example: "Yes, we used AI coding tools while building, like most developers do now. We decided what to build, gave it our lot photo and our design reference, tested everything, and we understand every part. Ask me about any file." This guide exists so that last sentence is true.

Q29. How did you make this presentation?

- The web deck is one HTML file, animated with GSAP (a JavaScript animation library), in the same fonts and colours as the site.
- The PowerPoint copy was generated from code with pptxgenjs.

Q30. Who did what? Give your one line from slide 16: "the site, the booking flow, dark mode and the bay pass". Then hand the next question to the right teammate:

- Kritika for the plan, lens and 3D
- Prisha for the server
- Krish for research and impact

16. Words you should know

WORD	MEANING
Frontend / backend	What runs in the browser / what runs on the server.
Component	A function that returns a piece of UI (`Ticket`, `SlotPanel`).
Props	The inputs you give a component, e.g. `<Ticket b={booking} s={bay} />`.
State	Data that can change. When it changes, React redraws.
Hook	A React function starting with `use`. `useState` holds state, `useEffect` runs code after drawing, `useMemo` remembers a result.
Context	React's way to share data with many components without passing it down by hand. `LotProvider` is our context.
Portal	Lets one React app draw into several places on the page. That's how one shared state feeds three islands.
Island	A small interactive React part inside a static page.
DOM	The browser's live model of the page that JavaScript can change.
API / endpoint	The set of URLs the server answers / one of those URLs, e.g. `POST /api/bookings`.
JSON	The text format data travels in: `{"bayId":"A-14","start":…}`.
HTTP status	The server's number reply: 200 OK, 400 bad request, **409 conflict** (already booked).
Cookie	A small value the browser keeps and sends back to the server; our anonymous device ID.
SSE (Server-Sent Events)	A connection that stays open so the server can push updates.
Bundle / build	Packing all source files into a few small files for the real site (`dist/`).
Dev server / hot reload	The local server while coding; the page updates on save.
Proxy	Forwarding: Vite passes `/api` requests to the Python server while we code.
Responsive / breakpoint	Layout that adapts to screen size / the width at which the layout changes.
Design token	A named design value, like `--sun: #ffd75a`.

WORD	MEANING
Easing	How an animation speeds up and slows down.
Linter	A tool that reads code and warns about mistakes.

17. Numbers to remember

Bays	**96** in **5 rows**: A 33 · B 10 · C 24 · D 6 · E 23
Prices	₹15 compact · ₹20 standard / step-free · ₹35 EV · + ₹5 fee
Ending soon	last **20 min** of a booking
Reserved	driver due within **30 min**
Grace	**15 min** past arrival
Day strip	**06:00 – 22:00**
Stay lengths offered	1, 2, 3, 4, 6, 8 h (the server allows 15 min to 12 h)
Live timings	clock every **1 s**, statuses every **5 s**, server ping every **15 s**, dead after **40 s**, reconnect 1 → 15 s
Code size	~**7,400** frontend lines · **1,121** backend lines
Built size	~**144 KB** JS + **13 KB** CSS (compressed)
Local addresses	`localhost:8787` (whole site) · `localhost:5173` (while coding)